ARCH_BREAKDOWN — obsidian-note-details-plugin

---

1. Structural Overview

1.1 File Tree

```
src/
├── main.ts                          entry point; NoteDetailsPlugin extends Plugin
├── settings.ts                      re-export shim
├── core/
│   ├── event-bus.ts                 type-safe pub/sub; BusEventMap defines all events
│   ├── layout-store.ts              pure functional helpers for preset/layout state
│   └── module-registry.ts           module lifecycle manager (register, loadAll, unloadAll)
├── interfaces/
│   ├── i-module.ts                  IModule contract; every module implements this
│   └── i-layout.ts                  HeightMode, ISlot, IPreset, IPluginLayout types
├── framework/
│   ├── control-panel.view.ts        main UI container; preset selector; fires active-file-ch
│   ├── layout-engine.ts             renders module widgets into the control panel
│   ├── preset-manager.ts            user-facing preset CRUD with callbacks
│   └── widget-host.ts               per-module wrapper; manages height/collapse chrome
├── settings/
│   ├── settings.ts                  PluginSettings interface + DEFAULT_SETTINGS
│   └── settings-tab.ts              settings UI (Obsidian PluginSettingTab)
└── modules/
    ├── note-anatomy/
    │   ├── note-anatomy.module.ts   IModule impl; wires vault reads → parser → view
    │   ├── note-anatomy.view.ts     tabbed UI (headings / URLs / cross-vault / frontmatter)
    │   └── markdown-parser.ts       stateless regex parsers; no Obsidian API dependency
    ├── connectivity/
    │   ├── connectivity.module.ts
    │   ├── connectivity.view.ts
    │   └── link-counter.ts          vault-wide link scoring (debounced)
    ├── focal-node/
    │   ├── focal-node.module.ts
    │   ├── focal-node.view.ts
    │   └── graph-builder.ts
    └── cross-vault/
        ├── cross-vault.module.ts
        ├── cross-vault.view.ts
        ├── vault-scanner.ts
        └── csv-exporter.ts
```

1.2 Bootstrap Sequence (`src/main.ts` → `onload()`)

```
1. new EventBus()              type-safe pub/sub singleton
2. new ModuleRegistry()        module lifecycle manager
3. loadSettings()              read persisted data; validate layout; reset if corrupted
4. registerView(ControlPanelView)
5. addRibbonIcon("layout-dashboard") → activateControlPanel()
6. addCommand("Open Control Panel")
7. registry.register(new FocalNodeModule())
   registry.register(new NoteAnatomyModule())
   registry.register(new ConnectivityModule())
   registry.register(new CrossVaultModule())
8. registry.loadAll(app, eventBus) → module.onload() called sequentially on each
```

```
9. addSettingTab(new NoteDetailsSettingTab())
```

```
onunload():
  registry.unloadAll() → module.onunload() on each
  eventBus.clear()
```

1.3 IModule Contract (`src/interfaces/i-module.ts`)

Member	Type	Purpose
`id`	`readonly string`	unique module key (e.g. `'note-anatomy'`)
`displayName`	`readonly string`	UI label
`onload(app, eventBus)`	`void`	init; subscribe to events; store app ref
`onunload()`	`void`	cleanup; remove listeners
`getView()`	`HTMLElement`	returns module's root DOM node
`onActiveFileChange(file)`	`void`	called on file switch
`onSettingsChange(settings)`	`void`	called on settings update

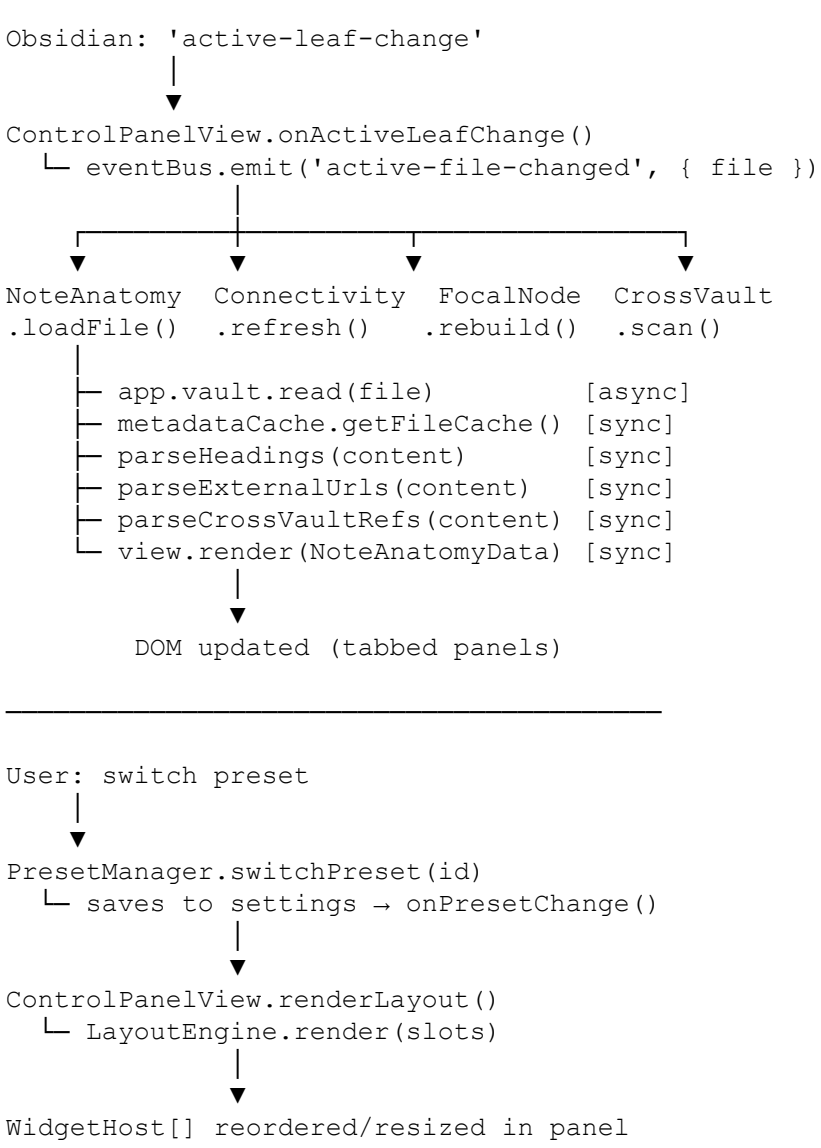
1.4 Framework Layer Components

Component	File	Role
`ControlPanelView`	framework/control-panel.view.ts	Obsidian ItemView; holds preset selector + LayoutEngine output; fires `active-file-changed` on workspace leaf change
`LayoutEngine`	framework/layout-engine.ts	Reads active preset → appends/reorders WidgetHost elements
`WidgetHost`	framework/widget-host.ts	Per-module chrome: height resize handle, collapse toggle, module label
`PresetManager`	framework/preset-manager.ts	CRUD: `createPreset`, `deletePreset`, `renamePreset`, `switchPreset`; invokes `onPresetChange` callback
`ModuleRegistry`	core/module-registry.ts	`Map<id, IModule>`; `loadAll` / `unloadAll` lifecycle
`EventBus`	core/event-bus.ts	`on(event, cb)` / `emit(event, payload)` / `off` / `clear`

1.5 EventBus Event Map

Event	Payload	Producer	Consumer(s)
`active-file-changed`	`{ file: TFile | null }`	ControlPanelView	all modules' `onload` listeners
`layout-preset-changed`	`{ presetId: string }`	PresetManager	ControlPanelView
`module-visibility`	`{ moduleId, visible }`	WidgetHost	ControlPanelView
`vault-scan-complete`	`{ stats }`	CrossVaultModule	any subscriber
`metadata-changed`	`{ file }`	ConnectivityModule	any subscriber

1.6 Data Flow (ASCII)



---

2. Note Anatomy Scraper — Technical Breakdown (Phase 1)

2.1 File Roles

File	Role
`markdown-parser.ts`	Stateless parsing engine. Three pure functions + two type defs. Zero Obsidian API usage.
`note-anatomy.module.ts`	IModule orchestrator. Owns vault/metadataCache reads, calls parsers, binds results to view.
`note-anatomy.view.ts`	Tabbed DOM component. Four panels: Headings, URLs, Cross-Vault Refs, Frontmatter. Handles editor navigation.

2.2 Exported Types & Signatures

`markdown-parser.ts`

```
interface ParsedHeading {
```

```
  level: 1 | 2 | 3;
  text: string;
  line: number;          // 0-indexed
}

interface ParsedCrossVaultRef {
  vault: string;
  file: string;
  raw: string;           // original obsidian:// URI
}

function parseHeadings(content: string): ParsedHeading[]
function parseExternalUrls(content: string): string[]
function parseCrossVaultRefs(content: string): ParsedCrossVaultRef[]
```

note-anatomy.view.ts

```
interface NoteAnatomyData {
  headings:       ParsedHeading[];
  urls:           string[];
  crossVaultRefs: ParsedCrossVaultRef[];
  frontmatter:    Record<string, unknown> | null;
  fileName:       string;
  filePath:       string;
}

class NoteAnatomyView {
  constructor(onNavigateToLine: (line: number) => void)
  render(data: NoteAnatomyData): void
  clear(): void
}
```

note-anatomy.module.ts

```
class NoteAnatomyModule implements IModule {
  readonly id          = 'note-anatomy'
  readonly displayName = 'Note Anatomy'
  onload(app: App, eventBus: EventBus): void
  onunload(): void
  getView(): HTMLElement
  onActiveFileChange(file: TFile | null): void
  onSettingsChange(_settings: PluginSettings): void  // no-op
}
```

2.3 Parsing Algorithm

Headings (`parseHeadings`)

```
1. Split content by '\n' → line array
2. Track fenced code blocks:
     toggle blockOpen on lines matching /^(```|~~~)/
     skip lines while blockOpen === true
3. Per non-code line, test regex: /^(#{1,3})\s+(.+)/
     match[1].length → level (1|2|3)   H4+ intentionally excluded
     match[2].trim() → text
     lineIndex       → line (0-indexed)
4. Push { level, text, line } to results[]
5. Return results[] — document order, no dedup, no sort
```

External URLs (`parseExternalUrls`)

```
1. Strip YAML frontmatter block (lines between first pair of '---' delimiters)
2. Apply global regex: /https?:\/\/[^\s\)\]\'"<>]+/g → rawMatches[]
3. Per match: trimTrailingPunct() — strip trailing chars in [.,;:!?)]
4. Insert into Set<string> for deduplication
5. Return Array.from(set) — encounter order, unique
```

Cross-Vault Refs (`parseCrossVaultRefs`)

```
1. Apply global regex: /obsidian:\/\/open\?[^\s\)\]\'"<>]+/g → rawMatches[]
2. Per match:
     try { url = new URL(match) } catch { skip }
     vault = decodeURIComponent(url.searchParams.get('vault') ?? '')
     file  = decodeURIComponent(url.searchParams.get('file')  ?? '')
     if (vault === '' && file === '') skip
3. Dedup by raw URI string (Set<string>)
4. Push { vault, file, raw } to results[]
5. Return results[] — encounter order
```

Frontmatter

```
- NOT parsed by markdown-parser.ts
- Module reads: app.metadataCache.getFileCache(file)?.frontmatter
- Strips internal Obsidian key: delete cache['position']
- Returns Record<string, unknown> | null
- Obsidian handles all YAML parsing (no regex)
```

2.4 Full Async Flow

```
EventBus fires 'active-file-changed'
    │
    ▼
NoteAnatomyModule listener
    └─ void this.loadFile(file)      [fire-and-forget async]
            │
            ├─ file.extension !== 'md' → view.clear(); return
            │
            ├─ await app.vault.read(file)         [I/O boundary]
            │      → content: string
            │
            ├─ app.metadataCache.getFileCache(file)?.frontmatter  [sync]
            │      → frontmatterRaw (may be partial/undefined on cold start)
            │      → delete frontmatterRaw['position']
            │
            ├─ parseHeadings(content)             [sync]
            ├─ parseExternalUrls(content)          [sync]
            ├─ parseCrossVaultRefs(content)        [sync]
            │
            └─ view.render({
                 headings, urls, crossVaultRefs,
                 frontmatter, fileName, filePath
               })                                  [sync DOM update]
```

2.5 Obsidian API Surfaces

Surface	Call	Usage
`App.vault`	`.read(file: TFile)`	Async read of raw markdown content
`App.metadataCache`	`.getFileCache(file)`	Sync access to parsed YAML frontmatter
`App.workspace`	`.getActiveFile()`	Load current file immediately at plugin startup
`App.workspace`	`.getActiveViewOfType(MarkdownView)`	Retrieve active editor for line navigation
`MarkdownView.editor`	`.setCursor(pos) .scrollIntoView(range, center) .focus()`	Navigate editor to heading line on click
`TFile`	`.extension .name .path`	Filter non-markdown files; display metadata

2.6 Key Constraints & Behaviors

Constraint	Detail
H4+ headings excluded	Parser regex `/^(#{1,3})/` — intentional; H4–H6 not surfaced
No re-parse caching	Every `active-file-changed` triggers full `vault.read` + all three parsers
No vault event hooks	Does not call `app.vault.on()` or `metadataCache.on()`; relies solely on EventBus
No external dependencies	Parsers use only native `String`, `Set`, `URL`, `RegExp`
Frontmatter delegated	YAML parsing is Obsidian's responsibility; module only reads the cache object
Code block exclusion	Heading parser skips lines inside ``` and ~~~ fenced blocks
XSS mitigations	URLs rendered with `target="_blank" rel="noopener noreferrer"`; frontmatter object values JSON-stringified
Settings integration	Registered as disabled by default (`'note-anatomy': false`); `onSettingsChange` is a no-op
Promise handling	`loadFile` is `async`; listener prefixes call with `void` (fire-and-forget; no error propagation)
Result ordering	All three parsers return results in document (encounter) order